



S.G. Ganesh

How Debugging Can Result In Bugs!

We typically debug code to find and fix bugs. However, debugging itself can cause bugs. This is an interesting phenomenon that we cover in this month's column.

Consider the following simple Java example, which I recently encountered:

```
Logger.log ("Skipping undefined data type: " + dataType.  
getCategory().toString() );
```

This is debugging code to print a log message that was meant for tracking what was going wrong in the program. This program crashed with a *NullPointerException* because the code did not check for null before accessing *dataType* or *getCategory()*. In other words, the very code that was meant for debugging, introduced a bug! This prompted me to write about the topic.

It is a good practice to save debug information and trace information in log files. When the application crashes at the customer's site, if the log files contain all the relevant information, it is possible to trace the cause of the problem by just analysing the log files.

However, a major problem in logging/tracing messages is that it generates huge amounts of data (sometimes, in the order of a few GBs!) and it is easy to get lost in the details. A practical approach is to introduce multiple trace levels, which is useful for trouble-shooting and debugging. In case of multiple threads, there needs to be some way of matching trace messages originating from a given thread of control (and process). Otherwise, if no such identification and time-stamping of trace messages is available, it will be difficult to make use of an exceptionally large trace file. For this reason, some projects develop custom scripts/tools to process the log files and report the problems! Also note that log files need to be removed periodically, if they grow beyond the 'allowed' size. I know about an application that used to crash often because the log files became so huge that no more data could be written to them.

Sometimes, when debugging code is added

to a system to understand how it is working, the debug code can introduce new bugs. For example, for diagnostic purposes, test probes can be added to get intermediate values at fixed locations in code. This process of introducing test probes can also bring with it subtle timing errors, particularly in the code of embedded systems, where response time is critical. In other words, the very process of examining the system, can alter it!

Debug code can also introduce security issues. In 1988, when the Internet was in the early stages of its development, a worm affected around 5 per cent of computers connected to the Internet—only Sun and VAX machines. It collected host, network and user information, based on which it broke into other machines. The affected machines were overloaded with unknown processes, and killing the processes did not help. Rebooting also didn't solve the problems. It was later found that the worm exploited three different vulnerabilities in Unix systems: a buffer overrun vulnerability in *fingerd*, the debug mode of the *sendmail* program, and accounts with weak (or no) passwords. Our interest here is in the attack on *sendmail* that exploited *debug* code. The worm would send a *DEBUG* command to *sendmail* and fork a shell (*sh*); it would use that shell to download and compile new worms, and thus spread the worm. Why did *sendmail* allow *DEBUG* code? It was provided to allow testers to verify that mail was arriving at a particular site, without the need to invoke the address-resolution routines! For more details about this worm, see the article, 'Crisis and Aftermath' by E H Spafford, Communications of the ACM, June 1989. 

By: S.G. Ganesh

S G Ganesh works for Siemens (Corporate Technology).
You can reach him at sgganesh@gmail.com.